dff9-W4111-Spring-2026 -HW3.ipynb

Introduction

Homework Overview

There is one version of the homework that both the programming and non-programming tracks complete.

There are 4 categories of question:

1. Short answer questions testing general knowledge and the ability to reason about the course material.

2. SQL questions testing the ability to use SQL to produce data results from descriptions of the desired output.

3. Questions testing the ability to use MongoDB to produce results from descriptions of the desired output.

4. Questions testing the ability to use Neo4j to produce results from the descriptions of the desired output.

Submission Instructions

The homework is due at 11:59 PM on Sunday, 18-April-2026.

Please see Ed discussion for detailed submission instructions and for clarifications and answering questions.

Part 0 — Setup

iPython-SQL

In [1]:
```
%load_ext sql
```

Set the proper root user ID and password for MySQL in the python cell below.

In [2]:
```
mysql_root_user = 'root'
mysql_root_password = 'david-database'

mysql_url = f"mysql+pymysql://{mysql_root_user}:{mysql_root_password}@
%pip install cryptography
```

```
Requirement already satisfied: cryptography in /Users/davidnguyen/Pycha
rmProjects/W4111-Intro-to-Databases-Spring-2026/venv/lib/python3.9/site
-packages (46.0.7)
Requirement already satisfied: cffi>=2.0.0 in /Users/davidnguyen/Pychar
mProjects/W4111-Intro-to-Databases-Spring-2026/venv/lib/python3.9/site-
packages (from cryptography) (2.0.0)
Requirement already satisfied: typing-extensions>=4.13.2 in /Users/davi
dnguyen/PycharmProjects/W4111-Intro-to-Databases-Spring-2026/venv/lib/p
ython3.9/site-packages (from cryptography) (4.15.0)
Requirement already satisfied: pycparser in /Users/davidnguyen/PycharmP
rojects/W4111-Intro-to-Databases-Spring-2026/venv/lib/python3.9/site-pa
ckages (from cffi>=2.0.0->cryptography) (2.23)

[notice] A new release of pip is available: 23.2.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

In [3]:
```
mysql_url
```

Out[3]:
```
'mysql+pymysql://root:david-database@localhost'
```

In [4]:
```
%sql $mysql_url
```

In [5]:
```
%config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

In [6]:
```
%sql select * from db_book.student where dept_name="Comp. Sci."
```

```
 * mysql+pymysql://root:***@localhost
4 rows affected.
```

Out[6]:

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
54321	Williams	Comp. Sci.	54
76543	Brown	Comp. Sci.	58

SQL Alchemy

In [7]:
```
from sqlalchemy import create_engine, text
```

In [8]:
```
engine = create_engine(mysql_url)
```

In [9]:
```
# Raw SQL query
sql = text("SELECT * FROM db_book.student WHERE dept_name = :dept")

# Execute query
with engine.connect() as connection:
    result = connection.execute(sql, {"dept": "Comp. Sci."})

    for row in result:
        print(row)
```

```
('00128', 'Zhang', 'Comp. Sci.', Decimal('102'))
('12345', 'Shankar', 'Comp. Sci.', Decimal('32'))
('54321', 'Williams', 'Comp. Sci.', Decimal('54'))
('76543', 'Brown', 'Comp. Sci.', Decimal('58'))
```

Pandas

In [10]:
```
import pandas
```

In [11]:
```
result = pandas.read_sql(con=engine, sql="select * from db_book.studen
```

In [12]:
```
result
```

Out[12]:

	ID	name	dept_name	tot_cred
0	00128	Zhang	Comp. Sci.	102.0
1	12345	Shankar	Comp. Sci.	32.0
2	54321	Williams	Comp. Sci.	54.0
3	76543	Brown	Comp. Sci.	58.0

PyMySQL

In [13]:
```
import pymysql
```

In [14]:
```
conn = pymysql.connect(
    host="localhost",
    user=mysql_root_user,
    password=mysql_root_password,
    cursorclass=pymysql.cursors.DictCursor,
    autocommit=True
)
```

In [15]:
```
cursor = conn.cursor()
```

In [16]:
```
result = cursor.execute("select * from db_book.student where dept_name
result = cursor.fetchall()
```

In [17]:
```
result
```

Out[17]:
```
[{'ID': '00128',
  'name': 'Zhang',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('102')},
 {'ID': '12345',
  'name': 'Shankar',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('32')},
 {'ID': '54321',
  'name': 'Williams',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('54')},
 {'ID': '76543',
  'name': 'Brown',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('58')}]
```

MongoDB

Test Connectivity

In [21]:
```
import pymongo
```

In [22]:
```
from pymongo import MongoClient, errors
```

In [23]:
```
import json
```

In [26]:
```
#
```

```
# You will need to change this URL to connect to the MongoDB instance
#
#mongourl = "mongodb+srv://dbuser:sBJ1btaxRAWiqfm1@cluster0.t8qdk.mong
mongourl = "mongodb+srv://gannetslumper2y_db_user:pfRAm13a80kHKZgT@clu
```

In [27]:
```
client = MongoClient(mongourl)
```

In [28]:
```
#
# Your answer wrt to databases and collections will be different based
# and collections you previously created. You connected successfully i
#

try:
    # Create a MongoClient with a short timeout for quick failure if u
    # client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)

    # Attempt to retrieve server information (forces a connection call
    server_info = client.server_info()
    print("✅ Successfully connected to MongoDB Atlas!")
    print("Server Info:", server_info.get("version", "Unknown version"

    # Optional: List available databases
    print("\nDatabases on this cluster:")
    for db_name in client.list_database_names():
        print(" -", db_name)

except errors.ServerSelectionTimeoutError as err:
    print("❌ Could not connect to MongoDB Atlas. Timeout occurred.")
    print("Error details:", err)
except errors.OperationFailure as err:
    print("❌ Authentication failed. Check your username/password.")
    print("Error details:", err)
except Exception as err:
    print("❌ An unexpected error occurred.")
    print("Error details:", err)
```

```
✅ Successfully connected to MongoDB Atlas!
Server Info: 8.0.21

Databases on this cluster:
 - admin
 - local
```

Load Classicmodels

In [29]:
```
import json
```

In [30]:
```
with open('customers.json', 'r') as in_file:
    customers = json.load(in_file)
```

In [31]:
```
customers[0:2]
```

Out[31]:
```
[{'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'contact': {'lastName': 'Schmitt', 'firstName': 'Carine '},
  'phone': '40.32.2555',
  'address': {'addressLine1': '54, rue Royale',
   'addressLine2': None,
   'city': 'Nantes',
   'state': None,
   'country': 'France',
   'postalCode': '44000'},
  'salesRepNumber': 1370,
  'creditLimit': 21000.0},
 {'customerNumber': 112,
  'customerName': 'Signal Gift Stores',
  'contact': {'lastName': 'King', 'firstName': 'Jean'},
  'phone': '7025551838',
  'address': {'addressLine1': '8489 Strong St.',
   'addressLine2': None,
   'city': 'Las Vegas',
   'state': 'NV',
   'country': 'USA',
   'postalCode': '83030'},
  'salesRepNumber': 1166,
  'creditLimit': 71800.0}]
```

In [32]:
```
len(customers)
```

Out[32]:
```
122
```

In [33]:
```
client['classicmodels']['customers'].insert_many(customers)
```

Out[33]:
```
InsertManyResult([ObjectId('69e807ee7b2c60635f1bfdb8'), ObjectId('69e
807ee7b2c60635f1bfdb9'), ObjectId('69e807ee7b2c60635f1bfdba'), Object
Id('69e807ee7b2c60635f1bfdbb'), ObjectId('69e807ee7b2c60635f1bfdbc'),
ObjectId('69e807ee7b2c60635f1bfdbd'), ObjectId('69e807ee7b2c60635f1bf
dbe'), ObjectId('69e807ee7b2c60635f1bfdbf'), ObjectId('69e807ee7b2c60
635f1bfdc0'), ObjectId('69e807ee7b2c60635f1bfdc1'), ObjectId('69e807e
e7b2c60635f1bfdc2'), ObjectId('69e807ee7b2c60635f1bfdc3'), ObjectId('
69e807ee7b2c60635f1bfdc4'), ObjectId('69e807ee7b2c60635f1bfdc5'), Obj
ectId('69e807ee7b2c60635f1bfdc6'), ObjectId('69e807ee7b2c60635f1bfdc
7'), ObjectId('69e807ee7b2c60635f1bfdc8'), ObjectId('69e807ee7b2c6063
5f1bfdc9'), ObjectId('69e807ee7b2c60635f1bfdca'), ObjectId('69e807ee7
b2c60635f1bfdcb'), ObjectId('69e807ee7b2c60635f1bfdcc'), ObjectId('69
e807ee7b2c60635f1bfdcd'), ObjectId('69e807ee7b2c60635f1bfdce'), Objec
tId('69e807ee7b2c60635f1bfdcf'), ObjectId('69e807ee7b2c60635f1bfdd
0'), ObjectId('69e807ee7b2c60635f1bfdd1'), ObjectId('69e807ee7b2c6063
5f1bfdd2'), ObjectId('69e807ee7b2c60635f1bfdd3'), ObjectId('69e807ee7
b2c60635f1bfdd4'), ObjectId('69e807ee7b2c60635f1bfdd5'), ObjectId('69
e807ee7b2c60635f1bfdd6'), ObjectId('69e807ee7b2c60635f1bfdd7'), Objec
tId('69e807ee7b2c60635f1bfdd8'), ObjectId('69e807ee7b2c60635f1bfdd
9'), ObjectId('69e807ee7b2c60635f1bfdda'), ObjectId('69e807ee7b2c6063
```

```
5f1bfddb'), ObjectId('69e807ee7b2c60635f1bfddc'), ObjectId('69e807ee7
b2c60635f1bfddd'), ObjectId('69e807ee7b2c60635f1bfdde'), ObjectId('69
e807ee7b2c60635f1bfddf'), ObjectId('69e807ee7b2c60635f1bfde0'), Objec
tId('69e807ee7b2c60635f1bfde1'), ObjectId('69e807ee7b2c60635f1bfde
2'), ObjectId('69e807ee7b2c60635f1bfde3'), ObjectId('69e807ee7b2c6063
5f1bfde4'), ObjectId('69e807ee7b2c60635f1bfde5'), ObjectId('69e807ee7
b2c60635f1bfde6'), ObjectId('69e807ee7b2c60635f1bfde7'), ObjectId('69
e807ee7b2c60635f1bfde8'), ObjectId('69e807ee7b2c60635f1bfde9'), Objec
tId('69e807ee7b2c60635f1bfdea'), ObjectId('69e807ee7b2c60635f1bfde
b'), ObjectId('69e807ee7b2c60635f1bfdec'), ObjectId('69e807ee7b2c6063
5f1bfded'), ObjectId('69e807ee7b2c60635f1bfdee'), ObjectId('69e807ee7
b2c60635f1bfdef'), ObjectId('69e807ee7b2c60635f1bfdf0'), ObjectId('69
e807ee7b2c60635f1bfdf1'), ObjectId('69e807ee7b2c60635f1bfdf2'), Objec
tId('69e807ee7b2c60635f1bfdf3'), ObjectId('69e807ee7b2c60635f1bfdf
4'), ObjectId('69e807ee7b2c60635f1bfdf5'), ObjectId('69e807ee7b2c6063
5f1bfdf6'), ObjectId('69e807ee7b2c60635f1bfdf7'), ObjectId('69e807ee7
b2c60635f1bfdf8'), ObjectId('69e807ee7b2c60635f1bfdf9'), ObjectId('69
e807ee7b2c60635f1bfdfa'), ObjectId('69e807ee7b2c60635f1bfdfb'), Objec
tId('69e807ee7b2c60635f1bfdfc'), ObjectId('69e807ee7b2c60635f1bfdf
d'), ObjectId('69e807ee7b2c60635f1bfdfe'), ObjectId('69e807ee7b2c6063
5f1bfdff'), ObjectId('69e807ee7b2c60635f1bfe00'), ObjectId('69e807ee7
b2c60635f1bfe01'), ObjectId('69e807ee7b2c60635f1bfe02'), ObjectId('69
e807ee7b2c60635f1bfe03'), ObjectId('69e807ee7b2c60635f1bfe04'), Objec
tId('69e807ee7b2c60635f1bfe05'), ObjectId('69e807ee7b2c60635f1bfe0
6'), ObjectId('69e807ee7b2c60635f1bfe07'), ObjectId('69e807ee7b2c6063
5f1bfe08'), ObjectId('69e807ee7b2c60635f1bfe09'), ObjectId('69e807ee7
b2c60635f1bfe0a'), ObjectId('69e807ee7b2c60635f1bfe0b'), ObjectId('69
e807ee7b2c60635f1bfe0c'), ObjectId('69e807ee7b2c60635f1bfe0d'), Objec
tId('69e807ee7b2c60635f1bfe0e'), ObjectId('69e807ee7b2c60635f1bfe0
f'), ObjectId('69e807ee7b2c60635f1bfe10'), ObjectId('69e807ee7b2c6063
5f1bfe11'), ObjectId('69e807ee7b2c60635f1bfe12'), ObjectId('69e807ee7
b2c60635f1bfe13'), ObjectId('69e807ee7b2c60635f1bfe14'), ObjectId('69
e807ee7b2c60635f1bfe15'), ObjectId('69e807ee7b2c60635f1bfe16'), Objec
tId('69e807ee7b2c60635f1bfe17'), ObjectId('69e807ee7b2c60635f1bfe1
8'), ObjectId('69e807ee7b2c60635f1bfe19'), ObjectId('69e807ee7b2c6063
5f1bfe1a'), ObjectId('69e807ee7b2c60635f1bfe1b'), ObjectId('69e807ee7
b2c60635f1bfe1c'), ObjectId('69e807ee7b2c60635f1bfe1d'), ObjectId('69
e807ee7b2c60635f1bfe1e'), ObjectId('69e807ee7b2c60635f1bfe1f'), Objec
tId('69e807ee7b2c60635f1bfe20'), ObjectId('69e807ee7b2c60635f1bfe2
1'), ObjectId('69e807ee7b2c60635f1bfe22'), ObjectId('69e807ee7b2c6063
5f1bfe23'), ObjectId('69e807ee7b2c60635f1bfe24'), ObjectId('69e807ee7
b2c60635f1bfe25'), ObjectId('69e807ee7b2c60635f1bfe26'), ObjectId('69
e807ee7b2c60635f1bfe27'), ObjectId('69e807ee7b2c60635f1bfe28'), Objec
tId('69e807ee7b2c60635f1bfe29'), ObjectId('69e807ee7b2c60635f1bfe2
a'), ObjectId('69e807ee7b2c60635f1bfe2b'), ObjectId('69e807ee7b2c6063
5f1bfe2c'), ObjectId('69e807ee7b2c60635f1bfe2d'), ObjectId('69e807ee7
b2c60635f1bfe2e'), ObjectId('69e807ee7b2c60635f1bfe2f'), ObjectId('69
e807ee7b2c60635f1bfe30'), ObjectId('69e807ee7b2c60635f1bfe31')], ackn
owledged=True)
```

In [34]:

```
with open("orders.json", "r") as in_file:
    orders = json.load(in_file)
```

In [36]:
```
orders[0:2]
```

Out[36]:
```
[{'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'orderDetails': [{'productCode': 'S18_1749',
    'quantityOrdered': 30,
    'priceEach': 136.0,
    'orderLineNumber': 3},
   {'productCode': 'S18_2248',
    'quantityOrdered': 50,
    'priceEach': 55.09,
    'orderLineNumber': 2},
   {'productCode': 'S18_4409',
    'quantityOrdered': 22,
    'priceEach': 75.46,
    'orderLineNumber': 4},
   {'productCode': 'S24_3969',
    'quantityOrdered': 49,
    'priceEach': 35.29,
    'orderLineNumber': 1}]},
 {'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128,
  'orderDetails': [{'productCode': 'S18_2325',
    'quantityOrdered': 25,
    'priceEach': 108.06,
    'orderLineNumber': 4},
   {'productCode': 'S18_2795',
    'quantityOrdered': 26,
    'priceEach': 167.06,
    'orderLineNumber': 1},
   {'productCode': 'S24_1937',
    'quantityOrdered': 45,
    'priceEach': 32.53,
    'orderLineNumber': 3},
   {'productCode': 'S24_2022',
    'quantityOrdered': 46,
    'priceEach': 44.35,
    'orderLineNumber': 2}]}]
```

In [37]:
```
len(orders)
```

Out[37]:
```
326
```

In [38]:
```
client['classicmodels']['orders'].insert_many(orders)
```

Out[38]:
```
InsertManyResult([ObjectId('69e808047b2c60635f1bfe32'), ObjectId('69e
808047b2c60635f1bfe33'), ObjectId('69e808047b2c60635f1bfe34'), Object
Id('69e808047b2c60635f1bfe35'), ObjectId('69e808047b2c60635f1bfe36'),
ObjectId('69e808047b2c60635f1bfe37'), ObjectId('69e808047b2c60635f1bf
e38'), ObjectId('69e808047b2c60635f1bfe39'), ObjectId('69e808047b2c60
635f1bfe3a'), ObjectId('69e808047b2c60635f1bfe3b'), ObjectId('69e8080
47b2c60635f1bfe3c'), ObjectId('69e808047b2c60635f1bfe3d'), ObjectId('
69e808047b2c60635f1bfe3e'), ObjectId('69e808047b2c60635f1bfe3f'), Obj
ectId('69e808047b2c60635f1bfe40'), ObjectId('69e808047b2c60635f1bfe4
1'), ObjectId('69e808047b2c60635f1bfe42'), ObjectId('69e808047b2c6063
5f1bfe43'), ObjectId('69e808047b2c60635f1bfe44'), ObjectId('69e808047
b2c60635f1bfe45'), ObjectId('69e808047b2c60635f1bfe46'), ObjectId('69
e808047b2c60635f1bfe47'), ObjectId('69e808047b2c60635f1bfe48'), Objec
tId('69e808047b2c60635f1bfe49'), ObjectId('69e808047b2c60635f1bfe4
a'), ObjectId('69e808047b2c60635f1bfe4b'), ObjectId('69e808047b2c6063
5f1bfe4c'), ObjectId('69e808047b2c60635f1bfe4d'), ObjectId('69e808047
b2c60635f1bfe4e'), ObjectId('69e808047b2c60635f1bfe4f'), ObjectId('69
e808047b2c60635f1bfe50'), ObjectId('69e808047b2c60635f1bfe51'), Objec
tId('69e808047b2c60635f1bfe52'), ObjectId('69e808047b2c60635f1bfe5
3'), ObjectId('69e808047b2c60635f1bfe54'), ObjectId('69e808047b2c6063
5f1bfe55'), ObjectId('69e808047b2c60635f1bfe56'), ObjectId('69e808047
b2c60635f1bfe57'), ObjectId('69e808047b2c60635f1bfe58'), ObjectId('69
e808047b2c60635f1bfe59'), ObjectId('69e808047b2c60635f1bfe5a'), Objec
tId('69e808047b2c60635f1bfe5b'), ObjectId('69e808047b2c60635f1bfe5
c'), ObjectId('69e808047b2c60635f1bfe5d'), ObjectId('69e808047b2c6063
5f1bfe5e'), ObjectId('69e808047b2c60635f1bfe5f'), ObjectId('69e808047
b2c60635f1bfe60'), ObjectId('69e808047b2c60635f1bfe61'), ObjectId('69
e808047b2c60635f1bfe62'), ObjectId('69e808047b2c60635f1bfe63'), Objec
tId('69e808047b2c60635f1bfe64'), ObjectId('69e808047b2c60635f1bfe6
5'), ObjectId('69e808047b2c60635f1bfe66'), ObjectId('69e808047b2c6063
5f1bfe67'), ObjectId('69e808047b2c60635f1bfe68'), ObjectId('69e808047
b2c60635f1bfe69'), ObjectId('69e808047b2c60635f1bfe6a'), ObjectId('69
e808047b2c60635f1bfe6b'), ObjectId('69e808047b2c60635f1bfe6c'), Objec
tId('69e808047b2c60635f1bfe6d'), ObjectId('69e808047b2c60635f1bfe6
e'), ObjectId('69e808047b2c60635f1bfe6f'), ObjectId('69e808047b2c6063
5f1bfe70'), ObjectId('69e808047b2c60635f1bfe71'), ObjectId('69e808047
b2c60635f1bfe72'), ObjectId('69e808047b2c60635f1bfe73'), ObjectId('69
e808047b2c60635f1bfe74'), ObjectId('69e808047b2c60635f1bfe75'), Objec
tId('69e808047b2c60635f1bfe76'), ObjectId('69e808047b2c60635f1bfe7
7'), ObjectId('69e808047b2c60635f1bfe78'), ObjectId('69e808047b2c6063
5f1bfe79'), ObjectId('69e808047b2c60635f1bfe7a'), ObjectId('69e808047
b2c60635f1bfe7b'), ObjectId('69e808047b2c60635f1bfe7c'), ObjectId('69
e808047b2c60635f1bfe7d'), ObjectId('69e808047b2c60635f1bfe7e'), Objec
tId('69e808047b2c60635f1bfe7f'), ObjectId('69e808047b2c60635f1bfe8
0'), ObjectId('69e808047b2c60635f1bfe81'), ObjectId('69e808047b2c6063
5f1bfe82'), ObjectId('69e808047b2c60635f1bfe83'), ObjectId('69e808047
b2c60635f1bfe84'), ObjectId('69e808047b2c60635f1bfe85'), ObjectId('69
e808047b2c60635f1bfe86'), ObjectId('69e808047b2c60635f1bfe87'), Objec
tId('69e808047b2c60635f1bfe88'), ObjectId('69e808047b2c60635f1bfe8
9'), ObjectId('69e808047b2c60635f1bfe8a'), ObjectId('69e808047b2c6063
```

5f1bfe8b'), ObjectId('69e808047b2c60635f1bfe8c'), ObjectId('69e808047
b2c60635f1bfe8d'), ObjectId('69e808047b2c60635f1bfe8e'), ObjectId('69
e808047b2c60635f1bfe8f'), ObjectId('69e808047b2c60635f1bfe90'), Objec
tId('69e808047b2c60635f1bfe91'), ObjectId('69e808047b2c60635f1bfe9
2'), ObjectId('69e808047b2c60635f1bfe93'), ObjectId('69e808047b2c6063
5f1bfe94'), ObjectId('69e808047b2c60635f1bfe95'), ObjectId('69e808047
b2c60635f1bfe96'), ObjectId('69e808047b2c60635f1bfe97'), ObjectId('69
e808047b2c60635f1bfe98'), ObjectId('69e808047b2c60635f1bfe99'), Objec
tId('69e808047b2c60635f1bfe9a'), ObjectId('69e808047b2c60635f1bfe9
b'), ObjectId('69e808047b2c60635f1bfe9c'), ObjectId('69e808047b2c6063
5f1bfe9d'), ObjectId('69e808047b2c60635f1bfe9e'), ObjectId('69e808047
b2c60635f1bfe9f'), ObjectId('69e808047b2c60635f1bfea0'), ObjectId('69
e808047b2c60635f1bfea1'), ObjectId('69e808047b2c60635f1bfea2'), Objec
tId('69e808047b2c60635f1bfea3'), ObjectId('69e808047b2c60635f1bfea
4'), ObjectId('69e808047b2c60635f1bfea5'), ObjectId('69e808047b2c6063
5f1bfea6'), ObjectId('69e808047b2c60635f1bfea7'), ObjectId('69e808047
b2c60635f1bfea8'), ObjectId('69e808047b2c60635f1bfea9'), ObjectId('69
e808047b2c60635f1bfeaa'), ObjectId('69e808047b2c60635f1bfeab'), Objec
tId('69e808047b2c60635f1bfeac'), ObjectId('69e808047b2c60635f1bfea
d'), ObjectId('69e808047b2c60635f1bfeae'), ObjectId('69e808047b2c6063
5f1bfeaf'), ObjectId('69e808047b2c60635f1bfeb0'), ObjectId('69e808047
b2c60635f1bfeb1'), ObjectId('69e808047b2c60635f1bfeb2'), ObjectId('69
e808047b2c60635f1bfeb3'), ObjectId('69e808047b2c60635f1bfeb4'), Objec
tId('69e808047b2c60635f1bfeb5'), ObjectId('69e808047b2c60635f1bfeb
6'), ObjectId('69e808047b2c60635f1bfeb7'), ObjectId('69e808047b2c6063
5f1bfeb8'), ObjectId('69e808047b2c60635f1bfeb9'), ObjectId('69e808047
b2c60635f1bfeba'), ObjectId('69e808047b2c60635f1bfebb'), ObjectId('69
e808047b2c60635f1bfebc'), ObjectId('69e808047b2c60635f1bfebd'), Objec
tId('69e808047b2c60635f1bfebe'), ObjectId('69e808047b2c60635f1bfeb
f'), ObjectId('69e808047b2c60635f1bfec0'), ObjectId('69e808047b2c6063
5f1bfec1'), ObjectId('69e808047b2c60635f1bfec2'), ObjectId('69e808047
b2c60635f1bfec3'), ObjectId('69e808047b2c60635f1bfec4'), ObjectId('69
e808047b2c60635f1bfec5'), ObjectId('69e808047b2c60635f1bfec6'), Objec
tId('69e808047b2c60635f1bfec7'), ObjectId('69e808047b2c60635f1bfec
8'), ObjectId('69e808047b2c60635f1bfec9'), ObjectId('69e808047b2c6063
5f1bfeca'), ObjectId('69e808047b2c60635f1bfecb'), ObjectId('69e808047
b2c60635f1bfecc'), ObjectId('69e808047b2c60635f1bfecd'), ObjectId('69
e808047b2c60635f1bfece'), ObjectId('69e808047b2c60635f1bfecf'), Objec
tId('69e808047b2c60635f1bfed0'), ObjectId('69e808047b2c60635f1bfed
1'), ObjectId('69e808047b2c60635f1bfed2'), ObjectId('69e808047b2c6063
5f1bfed3'), ObjectId('69e808047b2c60635f1bfed4'), ObjectId('69e808047
b2c60635f1bfed5'), ObjectId('69e808047b2c60635f1bfed6'), ObjectId('69
e808047b2c60635f1bfed7'), ObjectId('69e808047b2c60635f1bfed8'), Objec
tId('69e808047b2c60635f1bfed9'), ObjectId('69e808047b2c60635f1bfed
a'), ObjectId('69e808047b2c60635f1bfedb'), ObjectId('69e808047b2c6063
5f1bfedc'), ObjectId('69e808047b2c60635f1bfedd'), ObjectId('69e808047
b2c60635f1bfede'), ObjectId('69e808047b2c60635f1bfedf'), ObjectId('69
e808047b2c60635f1bfee0'), ObjectId('69e808047b2c60635f1bfee1'), Objec
tId('69e808047b2c60635f1bfee2'), ObjectId('69e808047b2c60635f1bfee
3'), ObjectId('69e808047b2c60635f1bfee4'), ObjectId('69e808047b2c6063
5f1bfee5'), ObjectId('69e808047b2c60635f1bfee6'), ObjectId('69e808047
b2c60635f1bfee7'), ObjectId('69e808047b2c60635f1bfee8'), ObjectId('69

e808047b2c60635f1bfee9'), ObjectId('69e808047b2c60635f1bfeea'), Objec
tId('69e808047b2c60635f1bfeeb'), ObjectId('69e808047b2c60635f1bfee
c'), ObjectId('69e808047b2c60635f1bfeed'), ObjectId('69e808047b2c6063
5f1bfeee'), ObjectId('69e808047b2c60635f1bfeef'), ObjectId('69e808047
b2c60635f1bfef0'), ObjectId('69e808047b2c60635f1bfef1'), ObjectId('69
e808047b2c60635f1bfef2'), ObjectId('69e808047b2c60635f1bfef3'), Objec
tId('69e808047b2c60635f1bfef4'), ObjectId('69e808047b2c60635f1bfef
5'), ObjectId('69e808047b2c60635f1bfef6'), ObjectId('69e808047b2c6063
5f1bfef7'), ObjectId('69e808047b2c60635f1bfef8'), ObjectId('69e808047
b2c60635f1bfef9'), ObjectId('69e808047b2c60635f1bfefa'), ObjectId('69
e808047b2c60635f1bfefb'), ObjectId('69e808047b2c60635f1bfefc'), Objec
tId('69e808047b2c60635f1bfefd'), ObjectId('69e808047b2c60635f1bfef
e'), ObjectId('69e808047b2c60635f1bfeff'), ObjectId('69e808047b2c6063
5f1bff00'), ObjectId('69e808047b2c60635f1bff01'), ObjectId('69e808047
b2c60635f1bff02'), ObjectId('69e808047b2c60635f1bff03'), ObjectId('69
e808047b2c60635f1bff04'), ObjectId('69e808047b2c60635f1bff05'), Objec
tId('69e808047b2c60635f1bff06'), ObjectId('69e808047b2c60635f1bff0
7'), ObjectId('69e808047b2c60635f1bff08'), ObjectId('69e808047b2c6063
5f1bff09'), ObjectId('69e808047b2c60635f1bff0a'), ObjectId('69e808047
b2c60635f1bff0b'), ObjectId('69e808047b2c60635f1bff0c'), ObjectId('69
e808047b2c60635f1bff0d'), ObjectId('69e808047b2c60635f1bff0e'), Objec
tId('69e808047b2c60635f1bff0f'), ObjectId('69e808047b2c60635f1bff1
0'), ObjectId('69e808047b2c60635f1bff11'), ObjectId('69e808047b2c6063
5f1bff12'), ObjectId('69e808047b2c60635f1bff13'), ObjectId('69e808047
b2c60635f1bff14'), ObjectId('69e808047b2c60635f1bff15'), ObjectId('69
e808047b2c60635f1bff16'), ObjectId('69e808047b2c60635f1bff17'), Objec
tId('69e808047b2c60635f1bff18'), ObjectId('69e808047b2c60635f1bff1
9'), ObjectId('69e808047b2c60635f1bff1a'), ObjectId('69e808047b2c6063
5f1bff1b'), ObjectId('69e808047b2c60635f1bff1c'), ObjectId('69e808047
b2c60635f1bff1d'), ObjectId('69e808047b2c60635f1bff1e'), ObjectId('69
e808047b2c60635f1bff1f'), ObjectId('69e808047b2c60635f1bff20'), Objec
tId('69e808047b2c60635f1bff21'), ObjectId('69e808047b2c60635f1bff2
2'), ObjectId('69e808047b2c60635f1bff23'), ObjectId('69e808047b2c6063
5f1bff24'), ObjectId('69e808047b2c60635f1bff25'), ObjectId('69e808047
b2c60635f1bff26'), ObjectId('69e808047b2c60635f1bff27'), ObjectId('69
e808047b2c60635f1bff28'), ObjectId('69e808047b2c60635f1bff29'), Objec
tId('69e808047b2c60635f1bff2a'), ObjectId('69e808047b2c60635f1bff2
b'), ObjectId('69e808047b2c60635f1bff2c'), ObjectId('69e808047b2c6063
5f1bff2d'), ObjectId('69e808047b2c60635f1bff2e'), ObjectId('69e808047
b2c60635f1bff2f'), ObjectId('69e808047b2c60635f1bff30'), ObjectId('69
e808047b2c60635f1bff31'), ObjectId('69e808047b2c60635f1bff32'), Objec
tId('69e808047b2c60635f1bff33'), ObjectId('69e808047b2c60635f1bff3
4'), ObjectId('69e808047b2c60635f1bff35'), ObjectId('69e808047b2c6063
5f1bff36'), ObjectId('69e808047b2c60635f1bff37'), ObjectId('69e808047
b2c60635f1bff38'), ObjectId('69e808047b2c60635f1bff39'), ObjectId('69
e808047b2c60635f1bff3a'), ObjectId('69e808047b2c60635f1bff3b'), Objec
tId('69e808047b2c60635f1bff3c'), ObjectId('69e808047b2c60635f1bff3
d'), ObjectId('69e808047b2c60635f1bff3e'), ObjectId('69e808047b2c6063
5f1bff3f'), ObjectId('69e808047b2c60635f1bff40'), ObjectId('69e808047
b2c60635f1bff41'), ObjectId('69e808047b2c60635f1bff42'), ObjectId('69
e808047b2c60635f1bff43'), ObjectId('69e808047b2c60635f1bff44'), Objec
tId('69e808047b2c60635f1bff45'), ObjectId('69e808047b2c60635f1bff4

```
6'), ObjectId('69e808047b2c60635f1bff47'), ObjectId('69e808047b2c6063
5f1bff48'), ObjectId('69e808047b2c60635f1bff49'), ObjectId('69e808047
b2c60635f1bff4a'), ObjectId('69e808047b2c60635f1bff4b'), ObjectId('69
e808047b2c60635f1bff4c'), ObjectId('69e808047b2c60635f1bff4d'), Objec
tId('69e808047b2c60635f1bff4e'), ObjectId('69e808047b2c60635f1bff4
f'), ObjectId('69e808047b2c60635f1bff50'), ObjectId('69e808047b2c6063
5f1bff51'), ObjectId('69e808047b2c60635f1bff52'), ObjectId('69e808047
b2c60635f1bff53'), ObjectId('69e808047b2c60635f1bff54'), ObjectId('69
e808047b2c60635f1bff55'), ObjectId('69e808047b2c60635f1bff56'), Objec
tId('69e808047b2c60635f1bff57'), ObjectId('69e808047b2c60635f1bff5
8'), ObjectId('69e808047b2c60635f1bff59'), ObjectId('69e808047b2c6063
5f1bff5a'), ObjectId('69e808047b2c60635f1bff5b'), ObjectId('69e808047
b2c60635f1bff5c'), ObjectId('69e808047b2c60635f1bff5d'), ObjectId('69
e808047b2c60635f1bff5e'), ObjectId('69e808047b2c60635f1bff5f'), Objec
tId('69e808047b2c60635f1bff60'), ObjectId('69e808047b2c60635f1bff6
1'), ObjectId('69e808047b2c60635f1bff62'), ObjectId('69e808047b2c6063
5f1bff63'), ObjectId('69e808047b2c60635f1bff64'), ObjectId('69e808047
b2c60635f1bff65'), ObjectId('69e808047b2c60635f1bff66'), ObjectId('69
e808047b2c60635f1bff67'), ObjectId('69e808047b2c60635f1bff68'), Objec
tId('69e808047b2c60635f1bff69'), ObjectId('69e808047b2c60635f1bff6
a'), ObjectId('69e808047b2c60635f1bff6b'), ObjectId('69e808047b2c6063
5f1bff6c'), ObjectId('69e808047b2c60635f1bff6d'), ObjectId('69e808047
b2c60635f1bff6e'), ObjectId('69e808047b2c60635f1bff6f'), ObjectId('69
e808047b2c60635f1bff70'), ObjectId('69e808047b2c60635f1bff71'), Objec
tId('69e808047b2c60635f1bff72'), ObjectId('69e808047b2c60635f1bff7
3'), ObjectId('69e808047b2c60635f1bff74'), ObjectId('69e808047b2c6063
5f1bff75'), ObjectId('69e808047b2c60635f1bff76'), ObjectId('69e808047
b2c60635f1bff77')], acknowledged=True)
```

Neo4j

In [40]:
```
import neo4j
```

In [41]:
```
#
# You must change to the information you downloaded when you created y
#
NEO4J_URI="Neo4j+ssc://b57b835c.databases.neo4j.io"
NEO4J_USERNAME="b57b835c"
NEO4J_PASSWORD="QSYDWfOpkFexJINS7AdbO3Zx0oqXSw8nH0suS0obicY"
NEO4J_DATABASE="b57b835c"
AURA_INSTANCEID="b57b835c"
AURA_INSTANCENAME="Free instance"
```

In [42]:
```
from neo4j import GraphDatabase

# URI examples: "neo4j://localhost", "neo4j+s://xxx.databases.neo4j.io

URL = NEO4J_URI

AUTH = (NEO4J_USERNAME, NEO4J_PASSWORD)
```

```
def run_query(q):


    with GraphDatabase.driver(NEO4J_URI, auth=AUTH) as driver:
        driver.verify_connectivity()

        records, summary, keys = driver.execute_query(
            q,
            database_=NEO4J_DATABASE,
        )

    # Loop through results and do something with them
    print("\nResult is")
    for record in records:
        print(record.data())  # obtain record as dict
```

In [43]:
```
run_query("MATCH (p:Person)-[:FOLLOWS]->(:Person) RETURN p.name AS nam
```

```
Result is
{'name': 'Paul Blythe'}
{'name': 'Angela Scope'}
{'name': 'James Thompson'}
```

Written Questions

W1

Question

For relationships in the ER Model, briefly explain the concepts of *degree* and *cardinality*.

Answer

W2

Question

Briefly explain the similarities and differences between *functions, procedures* and *triggers*.

Answer

W3

Question

Consider the table below and using your knowledge of the the db_book sample DB associated with the sample textbook, identify three functional dependencies in the table below.

	ID	name	dept_name	salary	building	budget
1	76766	Crick	Biology	72000.00	Watson	90000.00
2	10101	Srinivasan	Comp. Sci.	65000.00	Taylor	100000.00
3	45565	Katz	Comp. Sci.	75000.00	Taylor	100000.00
4	83821	Brandt	Comp. Sci.	92000.00	Taylor	100000.00
5	98345	Kim	Elec. Eng.	80000.00	Taylor	85000.00
6	12121	Wu	Finance	90000.00	Painter	120000.00
7	76543	Singh	Finance	80000.00	Painter	120000.00
8	32343	El Said	History	60000.00	Painter	50000.00
9	58583	Califieri	History	62000.00	Painter	50000.00
10	15151	Mozart	Music	40000.00	Packard	80000.00
11	22222	Einstein	Physics	95000.00	Watson	70000.00
12	33456	Gold	Physics	87000.00	Watson	70000.00

Answer

W4

Question

Briefly compare BCNF and 3NF.

Answer

W5

Question

What are some benefits of RAID-5 compared to RAID-0 and RAID-1?

Answer

W6

Question

For what types of query might columnar file organization be better than row oriented organization.

Answer

W7

Question

For what types of query is LRU replacement a very bad buffer replacement policy?

Answer

S8

Question

How many clustered indexes can a table have? Why?

Answer

S9

Question

Briefly explain the concept of *index selectivity* and how it relates to query

processing/optimization?

Answer

S10

Question

Consider the following query for the db_book sample database associated with the recommended textbook.

```
select
    *
from
    student join takes using(ID)
where
    student.dept_name = 'Comp. Sci.';
```

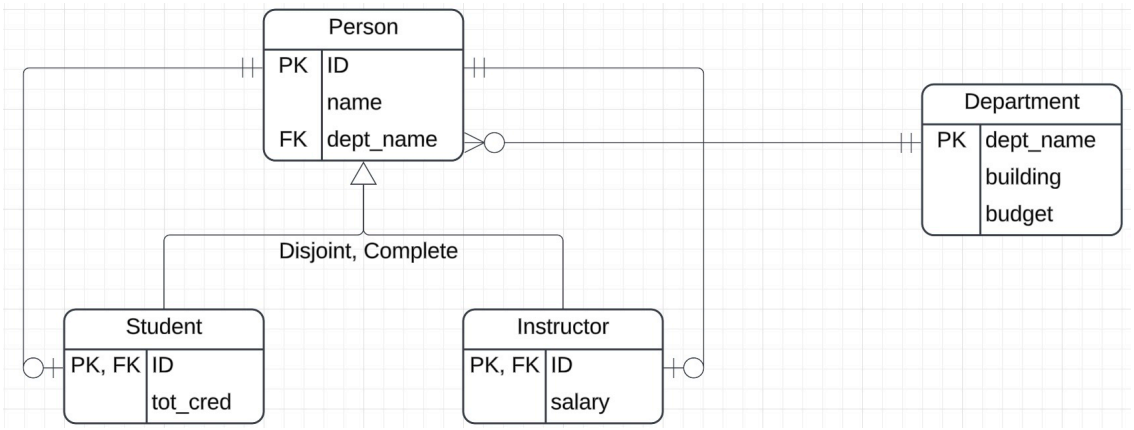
What is an equivalent query that might be more efficient?

Answer

SQL

S1

Consider the modification to the db_book schema depicted below. Write DDL create table and create view statements to implement the inheritance/specialization in SQL. Execute your statements in the cell below.



In [38]:
```
%%sql

drop schema if exists hw3;
```

```
 * mysql+pymysql://root:***@localhost
0 rows affected.
```

Out[38]: []

S2

Given your design above, write a function that computes a unique `ID` for a newly created `Person.` Execute your function and show the results.

In [39]:
```
%%sql
```

```
 * mysql+pymysql://root:***@localhost
```

S3

Consider the following table computed from the tables `student, takes` and `course` in db_book

ID	name	tot_cred	computed_credits	courses	grades	course_credits
00128	Zhang	102	7	CS-101,CS-347	A,A-	4,3
12345	Shankar	32	14	CS-101,CS-190,CS-315,CS-347	C,A,A,A	4,4,3,3
19991	Brandt	80	3	HIS-351	B	3
23121	Chavez	110	3	FIN-201	C+	3
44553	Peltier	56	4	PHY-101	B-	4
45678	Levy	46	7	CS-101,CS-101,CS-319	F,B+,B	0,4,3
54321	Williams	54	8	CS-101,CS-190	A-,B+	4,4
55739	Sanchez	38	3	MU-199	A-	3
70557	Snow	0	0	<null>	<null>	<null>
76543	Brown	58	7	CS-101,CS-319	A,A	4,3
76653	Aoi	60	3	EE-181	C	3
98765	Bourikas	98	7	CS-101,CS-315	C-,B	4,3
98988	Tanaka	120	4	BIO-101,BIO-301	A,I	4,0

The columns are computed using the following rules:

- `ID, name` and `tot_cred` come from `student`.
- `courses` is a list of the `course_ids` for the courses a student took. The

list is `null` if the student did not take any courses.

- `grades` is a list of grades the student earned in the courses and is computed from the following rules:
  - If the student did not take any course, the value is `null.`
  - If the student took a course but the grade was `null`, the value is `I`
  - Otherwise the value is the student's grade in the course.
- `course_credits` is a list of the `course.credits` for the courses a student took.
  - The list is `null` if the student did not take any courses.
  - The student earned a `0` in the course if the grade is `F` or `I`.
  - Otherwise the list contains the the credits for the course.

Write and execute SQL statement that produces the table.

MongoDB

Use the database and collections you created in the setup for these questions.

M1

Produce a list of `customers` in "France." Your answer should only contain the fields `customerNumber, customerName, address`.

In [11]:
```
#
# Set your filter expression and projection expression below.
#
filter = {}
projection = {}
```

In [12]:
```
#
# Execute this cell to perform the query.
#
result = client["classicmodels"]["customers"].find(
    filter = filter,
    projection = projection
)
result = list(result)
```

In [9]:
```
#
# Execute this cell to validate your result
#
len(result)
```

Out[9]: 12

In [13]:
```
#
# Execute this cell to display and validate your results.
#
result[0:2]
```

Out[13]:
```
[{'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'address': {'addressLine1': '54, rue Royale',
   'addressLine2': None,
   'city': 'Nantes',
   'state': None,
   'country': 'France',
   'postalCode': '44000'}},
 {'customerNumber': 119,
  'customerName': 'La Rochelle Gifts',
  'address': {'addressLine1': '67, rue des Cinquante Otages',
   'addressLine2': None,
   'city': 'Nantes',
   'state': None,
   'country': 'France',
   'postalCode': '44000'}}]
```

M2

Write a pipeline that produces a list of documents of the form:

- `orderNumber`
- `orderDate`
- `status`
- `requiredDate`
- `comments`
- `shippedDate`
- `customerNumber`
- `productCode`
- `quantityOrdered`
- `priceEach`
- `orderLineNumber`

Your list of documents should be sorted by `orderNumber` and `orderLineNumber`.

In [24]:
```
#
# Enter your pipeline here.
#
pipeline = [
```

```
]
```

In [25]:
```
#
# Execute this cell to run your query.
#
result = client['classicmodels']['orders'].aggregate(
    pipeline
)
result = list(result)
```

In [26]:
```
#
# Execute this cell to validate your query.
#
len(result)
```

Out[26]: 2996

In [27]:
```
#
# Execute this cell to validate and display your results.
#
result[0:20]
```

Out[27]:
```
[{'_id': ObjectId('69d2a88b6167f393b06c8e26'),
  'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S24_3969',
  'quantityOrdered': 49,
  'priceEach': 35.29,
  'orderLineNumber': 1},
 {'_id': ObjectId('69d2a88b6167f393b06c8e26'),
  'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S18_2248',
  'quantityOrdered': 50,
  'priceEach': 55.09,
  'orderLineNumber': 2},
 {'_id': ObjectId('69d2a88b6167f393b06c8e26'),
  'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
```

```
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 363,
 'productCode': 'S18_1749',
 'quantityOrdered': 30,
 'priceEach': 136.0,
 'orderLineNumber': 3},
{'_id': ObjectId('69d2a88b6167f393b06c8e26'),
 'orderNumber': 10100,
 'orderDate': '2003-01-06',
 'requiredDate': '2003-01-13',
 'shippedDate': '2003-01-10',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 363,
 'productCode': 'S18_4409',
 'quantityOrdered': 22,
 'priceEach': 75.46,
 'orderLineNumber': 4},
{'_id': ObjectId('69d2a88b6167f393b06c8e27'),
 'orderNumber': 10101,
 'orderDate': '2003-01-09',
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-11',
 'status': 'Shipped',
 'comments': 'Check on availability.',
 'customerNumber': 128,
 'productCode': 'S18_2795',
 'quantityOrdered': 26,
 'priceEach': 167.06,
 'orderLineNumber': 1},
{'_id': ObjectId('69d2a88b6167f393b06c8e27'),
 'orderNumber': 10101,
 'orderDate': '2003-01-09',
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-11',
 'status': 'Shipped',
 'comments': 'Check on availability.',
 'customerNumber': 128,
 'productCode': 'S24_2022',
 'quantityOrdered': 46,
 'priceEach': 44.35,
 'orderLineNumber': 2},
{'_id': ObjectId('69d2a88b6167f393b06c8e27'),
 'orderNumber': 10101,
 'orderDate': '2003-01-09',
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-11',
 'status': 'Shipped',
 'comments': 'Check on availability.',
 'customerNumber': 128,
 'productCode': 'S24_1937',
```

```
 'quantityOrdered': 45,
 'priceEach': 32.53,
 'orderLineNumber': 3},
{'_id': ObjectId('69d2a88b6167f393b06c8e27'),
 'orderNumber': 10101,
 'orderDate': '2003-01-09',
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-11',
 'status': 'Shipped',
 'comments': 'Check on availability.',
 'customerNumber': 128,
 'productCode': 'S18_2325',
 'quantityOrdered': 25,
 'priceEach': 108.06,
 'orderLineNumber': 4},
{'_id': ObjectId('69d2a88b6167f393b06c8e28'),
 'orderNumber': 10102,
 'orderDate': '2003-01-10',
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-14',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 181,
 'productCode': 'S18_1367',
 'quantityOrdered': 41,
 'priceEach': 43.13,
 'orderLineNumber': 1},
{'_id': ObjectId('69d2a88b6167f393b06c8e28'),
 'orderNumber': 10102,
 'orderDate': '2003-01-10',
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-14',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 181,
 'productCode': 'S18_1342',
 'quantityOrdered': 39,
 'priceEach': 95.55,
 'orderLineNumber': 2},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S24_2300',
 'quantityOrdered': 36,
 'priceEach': 107.34,
 'orderLineNumber': 1},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
```

```
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_2432',
 'quantityOrdered': 22,
 'priceEach': 58.34,
 'orderLineNumber': 2},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S32_1268',
 'quantityOrdered': 31,
 'priceEach': 92.46,
 'orderLineNumber': 3},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S10_4962',
 'quantityOrdered': 42,
 'priceEach': 119.67,
 'orderLineNumber': 4},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_4600',
 'quantityOrdered': 36,
 'priceEach': 98.07,
 'orderLineNumber': 5},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
```

```
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S700_2824',
 'quantityOrdered': 42,
 'priceEach': 94.07,
 'orderLineNumber': 6},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S32_3522',
 'quantityOrdered': 45,
 'priceEach': 63.35,
 'orderLineNumber': 7},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S12_1666',
 'quantityOrdered': 27,
 'priceEach': 121.64,
 'orderLineNumber': 8},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_4668',
 'quantityOrdered': 41,
 'priceEach': 40.75,
 'orderLineNumber': 9},
{'_id': ObjectId('69d2a88b6167f393b06c8e29'),
 'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_1097',
```

```
 'quantityOrdered': 35,
 'priceEach': 94.5,
 'orderLineNumber': 10}]
```

M3

The value of a line in `orderDetails` is `priceEach*quantityOrdered`.

The `totalValue` of an `order` is the sum over all line items of the value of the line.

Compute a list of documents of the form:

- `orderNumber`
- `totalValue`

Your answered should be sorted by `orderNumber.`

In [32]:
```
#
# Enter your pipeline here.
#
pipeline = [

]
```

In [33]:
```
#
# Execute this cell to run your query.
#
result = client['classicmodels']['orders'].aggregate(
    pipeline
)
result = list(result)
```

In [34]:
```
#
# Execute this cell to validate your query.
#
len(result)
```

Out[34]: 326

In [35]:
```
#
# Execute this cell to validate and display your results.
#
result[0:20]
```

Out[35]:
```
[{'orderNumber': 10100, 'totalValue': 10223.83},
 {'orderNumber': 10101, 'totalValue': 10549.01},
 {'orderNumber': 10102, 'totalValue': 5494.78},
 {'orderNumber': 10103, 'totalValue': 50218.95},
 {'orderNumber': 10104, 'totalValue': 40206.2},
 {'orderNumber': 10105, 'totalValue': 53959.21},
 {'orderNumber': 10106, 'totalValue': 52151.81},
 {'orderNumber': 10107, 'totalValue': 22292.62},
 {'orderNumber': 10108, 'totalValue': 51001.22},
 {'orderNumber': 10109, 'totalValue': 25833.14},
 {'orderNumber': 10110, 'totalValue': 48425.69},
 {'orderNumber': 10111, 'totalValue': 16537.85},
 {'orderNumber': 10112, 'totalValue': 7674.94},
 {'orderNumber': 10113, 'totalValue': 11044.3},
 {'orderNumber': 10114, 'totalValue': 33383.14},
 {'orderNumber': 10115, 'totalValue': 21665.98},
 {'orderNumber': 10116, 'totalValue': 1627.56},
 {'orderNumber': 10117, 'totalValue': 44380.15},
 {'orderNumber': 10118, 'totalValue': 3101.4},
 {'orderNumber': 10119, 'totalValue': 35826.33}]
```

Neo4j

Use Neo4j and the sample Movie Data for these queries.

N1

Write a query that returns the names of people who directed movies and the title of the movies they directed.

In [59]:
```
q = """
```

In [10]:
```
run_query(q)
```

```
Result is
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
"Rosie O'Donnell"}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Bill Paxton'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Madonna'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Geena Davis'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Lori Petty'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Kevin Baco
n'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Gary Sinis
```

e'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Ed Harris'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Bill Paxto
n'}
{'p.name': 'Tom Hanks', 'm.title': 'Cast Away', 'p2.name': 'Helen Hun
t'}
{'p.name': 'Tom Hanks', 'm.title': "Charlie Wilson's War", 'p2.name': '
Philip Seymour Hoffman'}
{'p.name': 'Tom Hanks', 'm.title': "Charlie Wilson's War", 'p2.name': '
Julia Roberts'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Hugo Weav
ing'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Halle Ber
ry'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Jim Broad
bent'}
{'p.name': 'Tom Hanks', 'm.title': 'Joe Versus the Volcano', 'p2.name':
'Meg Ryan'}
{'p.name': 'Tom Hanks', 'm.title': 'Joe Versus the Volcano', 'p2.name':
'Nathan Lane'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Meg Ryan'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Rita Wilson'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Bill Pullman'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Victor Garber'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': "
Rosie O'Donnell"}
{'p.name': 'Tom Hanks', 'm.title': 'That Thing You Do', 'p2.name': 'Cha
rlize Theron'}
{'p.name': 'Tom Hanks', 'm.title': 'That Thing You Do', 'p2.name': 'Liv
Tyler'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Ian
McKellen'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Aud
rey Tautou'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Pau
l Bettany'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Bonnie
Hunt'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'James
Cromwell'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Michae
l Clarke Duncan'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'David
Morse'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Sam Ro
ckwell'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Gary S
inise'}

```
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Patric
ia Clarkson'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Meg R
yan'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Greg
Kinnear'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Parke
r Posey'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Dave
Chappelle'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Steve
Zahn'}
```

N2

Write a query that computes the director_name, movie_title, and actor_name for each movie and order by director name.

In [61]:
```
q = """

"""
```

In [9]:
```
run_query(q)
```

```
Result is
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
"Rosie O'Donnell"}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Bill Paxton'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Madonna'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Geena Davis'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Lori Petty'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Kevin Baco
n'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Gary Sinis
e'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Ed Harris'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Bill Paxto
n'}
{'p.name': 'Tom Hanks', 'm.title': 'Cast Away', 'p2.name': 'Helen Hun
t'}
{'p.name': 'Tom Hanks', 'm.title': "Charlie Wilson's War", 'p2.name': '
Philip Seymour Hoffman'}
{'p.name': 'Tom Hanks', 'm.title': "Charlie Wilson's War", 'p2.name': '
Julia Roberts'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Hugo Weav
ing'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Halle Ber
```

ry'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Jim Broad
bent'}
{'p.name': 'Tom Hanks', 'm.title': 'Joe Versus the Volcano', 'p2.name':
'Meg Ryan'}
{'p.name': 'Tom Hanks', 'm.title': 'Joe Versus the Volcano', 'p2.name':
'Nathan Lane'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Meg Ryan'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Rita Wilson'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Bill Pullman'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Victor Garber'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': "
Rosie O'Donnell"}
{'p.name': 'Tom Hanks', 'm.title': 'That Thing You Do', 'p2.name': 'Cha
rlize Theron'}
{'p.name': 'Tom Hanks', 'm.title': 'That Thing You Do', 'p2.name': 'Liv
Tyler'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Ian
McKellen'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Aud
rey Tautou'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Pau
l Bettany'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Bonnie
Hunt'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'James
Cromwell'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Michae
l Clarke Duncan'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'David
Morse'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Sam Ro
ckwell'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Gary S
inise'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Patric
ia Clarkson'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Meg R
yan'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Greg
Kinnear'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Parke
r Posey'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Dave
Chappelle'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Steve
Zahn'}

N3

Return a list of all people who acted in a movie with Tim Hanks. Order the list by movie.title.

In [7]:
```
q = """

"""
```

In [8]:
```
run_query(q)
```

```
Result is
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
"Rosie O'Donnell"}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Bill Paxton'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Madonna'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Geena Davis'}
{'p.name': 'Tom Hanks', 'm.title': 'A League of Their Own', 'p2.name':
'Lori Petty'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Kevin Baco
n'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Gary Sinis
e'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Ed Harris'}
{'p.name': 'Tom Hanks', 'm.title': 'Apollo 13', 'p2.name': 'Bill Paxto
n'}
{'p.name': 'Tom Hanks', 'm.title': 'Cast Away', 'p2.name': 'Helen Hun
t'}
{'p.name': 'Tom Hanks', 'm.title': "Charlie Wilson's War", 'p2.name': '
Philip Seymour Hoffman'}
{'p.name': 'Tom Hanks', 'm.title': "Charlie Wilson's War", 'p2.name': '
Julia Roberts'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Hugo Weav
ing'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Halle Ber
ry'}
{'p.name': 'Tom Hanks', 'm.title': 'Cloud Atlas', 'p2.name': 'Jim Broad
bent'}
{'p.name': 'Tom Hanks', 'm.title': 'Joe Versus the Volcano', 'p2.name':
'Meg Ryan'}
{'p.name': 'Tom Hanks', 'm.title': 'Joe Versus the Volcano', 'p2.name':
'Nathan Lane'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Meg Ryan'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Rita Wilson'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Bill Pullman'}
```

```
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': '
Victor Garber'}
{'p.name': 'Tom Hanks', 'm.title': 'Sleepless in Seattle', 'p2.name': "
Rosie O'Donnell"}
{'p.name': 'Tom Hanks', 'm.title': 'That Thing You Do', 'p2.name': 'Cha
rlize Theron'}
{'p.name': 'Tom Hanks', 'm.title': 'That Thing You Do', 'p2.name': 'Liv
Tyler'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Ian
McKellen'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Aud
rey Tautou'}
{'p.name': 'Tom Hanks', 'm.title': 'The Da Vinci Code', 'p2.name': 'Pau
l Bettany'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Bonnie
Hunt'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'James
Cromwell'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Michae
l Clarke Duncan'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'David
Morse'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Sam Ro
ckwell'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Gary S
inise'}
{'p.name': 'Tom Hanks', 'm.title': 'The Green Mile', 'p2.name': 'Patric
ia Clarkson'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Meg R
yan'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Greg
Kinnear'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Parke
r Posey'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Dave
Chappelle'}
{'p.name': 'Tom Hanks', 'm.title': "You've Got Mail", 'p2.name': 'Steve
Zahn'}
```

In [ ]: